

Table des Matieres

1. Resume Executif	3
2. Infrastructure	3
2.1 PostgreSQL	3
2.2 Redis	4
2.3 Variables Environnement	4
3. Audit API	4
3.1 Vue d'ensemble	4
3.2 Problemes critiques API	4
3.3 Validation Zod manquante	5
4. Securite	5
4.1 Authentification	5
4.2 CSRF et CORS	6
4.3 RBAC	6
4.4 Chiffrement	6
4.5 Points positifs	6
5. Base de Donnees	6
5.1 Schema Prisma	6
5.2 Relations sans onDelete	6
5.3 Champs d'audit manquants	7
5.4 Index manquants	7

6. Frontend	7
6.1 Stores Zustand	7
6.2 Error Boundaries et PWA	7
6.3 il8n	8
7. Integrations	8
8. Build et Tests	8
9. Plan d'Action Priorise	8
9.1 Actions immediates (avant production)	8
9.2 Actions a court terme (1 sprint)	9
9.3 Actions a moyen terme (backlog)	9

1. Resume Executif

HealthFlow Guinea est un Systeme d'Information Hospitalier (SIH) developpe par DataSphere Innovation (Sekouna KABA), destine aux etablissements de sante de Guinee. L'application est construite sur Next.js 16.1.3 avec React 19, TypeScript, Tailwind CSS, shadcn/ui, et Prisma ORM connecte a PostgreSQL 17. L'audit end-to-end couvre l'infrastructure, la securite, les API, la base de donnees, le frontend et les integrations.

Synthese des resultats

Domaine	Statut	Problemes critiques
Infrastructure	Fonctionnel	Secrets en dur, env.example obsolete
API (55 routes)	A risque	40+ routes sans auth, 40+ sans validation Zod
Securite	A risque	JWT fallback divergent, RBAC non applique, CORS bypass
Base de donnees	Fonctionnel	42 relations sans onDelete, 0 enums Prisma
Frontend	Partiel	Stores localStorage, pas de error boundaries, i18n incomplete
Integrations	Partiel	Mode sandbox/demo uniquement, webhooks non verifiees
Build & Tests	Operationnel	Build OK, 352 tests passent

Tableau 1 : Synthese des resultats par domaine

Repartition des problemes par severite

Severite	Nombre	Exemples cles
CRITIQUE	3	JWT fallback, routes sans auth, pas de RBAC
HAUT	8	CORS bypass, validation manquante, ENCRYPTION_KEY placeholder
MOYEN	12	Rate limiting memoire, erreur leakage, stores localStorage
BAS	6	CSRF dev bypass, IV length, console.log残留
INFO	25	Bonne implementation OTP, bcryptjs, headers securite

Tableau 2 : Repartition des problemes par severite

2. Infrastructure

2.1 PostgreSQL

PostgreSQL 17 fonctionne correctement sur le port 5433 avec la base healthflow (67 tables, 12 Mo). La connexion Prisma est operationnelle. L'authentification est en mode trust

(sans mot de passe), acceptable uniquement pour le developpement local. Le seed minimalcharge 5 patients et 5 utilisateurs, laissant 50+ tables vides. Un probleme critique a ete identifie : la variable d'environnement systeme DATABASE_URL pointait vers SQLite(file:/home/z/my-project/db/custom.db), ecrasant la valeur PostgreSQL du fichier .env. Ce probleme a ete corrige dans db.ts et instrumentation.ts avec une detection et override automatique.

2.2 Redis

Redis 8.0.2 fonctionne sur le port 6380 avec 767 Ko de memoire utilisee. Le keyspace est vide (0 cles), indiquant qu'aucune session ou OTP n'a ete stocke recemment. Le RedisRateLimiter existe dans redis.ts mais n'est PAS utilise par le middleware, qui utilise un Map en memoire. En deploiement multi-instance, chaque instance aura son propre compteur de rate limiting, permettant a un attaquant de multiplier son budget.

2.3 Variables Environnement

Le fichier .env contient des secrets en dur qui doivent etre regeneres avant toute mise en production. Le fichier .env.example est obsolete avec des ports incorrects (5432 au lieu de 5433, 6379 au lieu de 6380) et omet JWT_SECRET. Les variables critiques identifiees sont les suivantes :

Variable	Probleme	Severite	Action
NEXTAUTH_SECRET	Placeholder faible	CRITIQUE	openssl rand -base64 32
JWT_SECRET	Placeholder faible	CRITIQUE	openssl rand -base64 32
ENCRYPTION_KEY	0123456789abcdef...	CRITIQUE	openssl rand -hex 32
DEMO_MODE	true desactive auth	HAUT	Mettre false en prod
.env.example	Ports incorrects	HAUT	Mettre a jour 5433/6380

Tableau 3 : Problemes de variables environnement

3. Audit API

3.1 Vue d'ensemble

L'application compte 55 fichiers de route API avec environ 125 gestionnaires HTTP. Seuls 5 fichiers (14 gestionnaires) utilisent secureApiHandler. Environ 46 fichiers (105 gestionnaires) n'ont AUCUNE authentication. L'audit a revele 3 problemes critiques, 8 de severite haute et 12 de severite moyenne.

3.2 Problemes critiques API

CRIT-1 : Divergence JWT_SECRET

Le fichier patient-auth/me/route.ts utilise un fallback JWT_SECRET different de tous les autres endpoints d'authentification ('healthflow-guinea-jwt-secret-key-2024' vs 'healthflow-guinea-jwt-secret-dev-only-NOT-FOR-PRODUCTION'). Si JWT_SECRET n'est pas defini dans l'environnement, les tokens emis par /patient-auth/verify ne peuvent PAS etre verifiees par /patient-auth/me, causant un echec complet de l'authentification du portail patient.

CRIT-2 : ~40 endpoints CRUD sans authentication

Les endpoints sensibles suivants n'ont aucune authentication : /api/patients (donnees PHI completes), /api/consultations, /api/billing, /api/payments, /api/hospitalizations, /api/laboratory, /api/emergencies, /api/pharmacy, /api/maternity, /api/vaccinations, /api/alerts, /api/establishments, /api/users, /api/fhir/*, /api/messaging/sms, /api/messaging/whatsapp. Toute requete non authentifiee peut lire, creer, modifier ou supprimer des donnees de sante protegees.

CRIT-3 : Webhook WhatsApp sans verification

Le endpoint GET /api/messaging/whatsapp ne valide jamais le hub.verify_token. Toute requete avec hub.mode=subscribe et un challenge sera acceptee, permettant a un attaquant d'enregistrer des URLs de webhook arbitraires.

3.3 Validation Zod manquante

Des schemas Zod existent dans src/lib/validations/ mais seulement ~12 des 50+ endpoints mutables les utilisent. Les endpoints critiques sans validation incluent : consultations, emergencies, hospitalizations, laboratory, pharmacy, billing, payments, maternity, vaccinations, insurance, messaging/sms, messaging/whatsapp, fhir/*, integrations, telemedicine/*, ai/*, et patient-auth/me (PUT). L'endpoint /api/patient-auth/me PUT passe les donnees directement a db.patient.update() sans AUCUNE validation de schema.

4. Securite

4.1 Authentification

L'authentification presente des fondations solides (bcryptjs 12 rounds, crypto.randomInt pour OTP, compteur de verrouillage apres 5 echecs, JWT HS256 avec expiration 12h) mais des lacunes critiques dans l'application homogene. Les IDs d'utilisateurs invites utilisent guest-\${Date.now()} qui est previsible. Le fallback NEXTAUTH_SECRET dans auth.ts est un string statique potentiellement mal configure.

4.2 CSRF et CORS

La protection CSRF est implementee en dual (middleware origin/referer + double-submit cookie). Cependant, `isAllowedOrigin()` utilise `.includes()` pour les sous-domaines, permettant a `evil-space.chatglm.site.attacker.com` de matcher `.space.chatglm.site`. De plus, `corsHeaders()` dans `api-utils.ts` revient silencieusement a `allowedOrigins[0]` quand l'origine n'est pas autorisee, accordant potentiellement un acces CORS involontaire.

4.3 RBAC

Le systeme RBAC est complet (8 roles, 30 permissions, scope `ownDataOnly` pour les patients) mais n'est PAS applique sur les routes API. Seul `/api/audit` utilise `secureApiHandler` avec RBAC. Le module RLS (Row-Level Security) dans `rls.ts` n'est importe dans AUCUNE route. `DEMO_MODE=true` fait confiance aux headers `x-user-id/x-user-role` du client, permettant l'usurpation d'identite si cette configuration fuite en production.

4.4 Chiffrement

L'implementation AES-256-GCM est correcte (IV 12 octets, auth tag verifie). Cependant, `security.ts` contient un fallback `ENCRYPTION_KEY` en dur ('`healthflow-guinea-32byte-encrypt-k`'), et le `.env` contient le placeholder `0123456789abcdef...`. Ces cles compromettraient toutes les donnees chiffrees si utilisees en production. Le module de sanitization XSS (`sanitizeInput/sanitizeObject`) existe mais n'est JAMAIS appele dans aucune route API.

4.5 Points positifs

Plusieurs mecanismes de securite sont bien implements : OTP avec `crypto.randomInt` (pas `Math.random`), stockage Redis avec TTL 5min et usage unique, OTP jamais retourne dans les reponses API, headers de securite complets (X-Frame-Options DENY, HSTS 1 an, CSP nonce en production, X-Content-Type-Options nosniff), anti-enumeration sur login, hash `bcryptjs` 12 rounds, et audit logging persiste en PostgreSQL avec buffer Redis.

5. Base de Donnees

5.1 Schema Prisma

Le schema comporte 45 modeles, ~89 relations, 66 indexes et 22 contraintes uniques. Zero enums Prisma sont definis ; les 56 champs de type enum sont stockes en String brut, sans validation au niveau base. Cela permet des valeurs invalides et des incoherences.

5.2 Relations sans onDelete

42 relations n'ont pas de regle onDelete, creant un risque d'enregistrements orphelins. Les plus dangereuses : Establishment vers Patient/Department/Room/Bed (suppression d'un hopital orphelinne tous les patients), User vers Consultation/LabRequest (suppression d'un medecin orphelinne les consultations), et Medication vers StockEntryItem/StockExitItem.

5.3 Champs d'audit manquants

Aucun modele n'a de champs createdBy/updatedBy. L'AuditLog capture qui a effectue les actions, mais les enregistrements individuels (Patient, Consultation, Prescription) ne suivent pas qui les a crees ou modifies. C'est un deficit de conformite majeur pour un systeme de dossiers medicaux.

5.4 Index manquants

Les colonnes establishmentId de 7+ modeles (Appointment, Consultation, Admission, EmergencyCase, LabRequest, PregnancyTracking, Notification) n'ont pas d'index, alors qu'elles sont filtrees dans presque toutes les requetes. Cela cause des scans sequentielles sur des tables qui grossiront en production.

6. Frontend

6.1 Stores Zustand

Les 4 stores Zustand utilisent tous localStorage avec zustand/persist, ce qui signifie que toutes les donnees medicales sont stockees dans le navigateur. AUCUN store ne fait d'appel API reel au backend. Le data-store.ts contient ~700 lignes de donnees demo en dur en francais. Les tokens d'authentification dans localStorage sont vulnerables aux attaques XSS. La migration vers des appels API reels est la priorite numero 1 du frontend.

Store	localStorage	Migration DB	Risque
auth-store.ts	Oui	Requise	Token XSS
patient-auth-store.ts	Oui	Requise	Token XSS
data-store.ts	Oui	Requise	Donnees medicales
hospital-store.ts	Oui	Requise	Donnees hopital

Tableau 4 : Etat des stores Zustand

6.2 Error Boundaries et PWA

Il n'existe AUCUN fichier error.tsx dans l'application. Tout crash de composant dynamique fait planter l'ensemble de l'application. Le manifest.json est complet mais les icones PWA dans /public/icons/ sont absentes (repertoire vide), rendant l'installation PWA

impossible. Le service worker est correctement implemente avec cache strategies, syncqueue et push notifications.

6.3 i18n

L'application supporte 5 langues mais les traductions sont inegales : Francais 100%, Anglais 100%, Maninka 37%, Fulfulde 27%, Susu 27%. De nombreuses chaines sont en dur en francais dans les stores, le service worker et le manifest.json. Les locuteurs des langues autochtones ne peuvent pas utiliser le systeme de maniere adequate.

7. Integrations

Toutes les integrations sont en mode sandbox/demo car les credentials API ne sont pas configurees. Orange Money et MTN MoMo sont implementees avec OAuth2, HMAC-SHA256 pour les webhooks, et auto-detection du provider par prefixe telefonique. DHIS2 supporte la collecte de donnees depuis PostgreSQL et la soumission DataValueSets. DICOM/Orthanc gere le multi-serveur PACS. WhatsApp Business supporte Twilio et Meta Cloud API avec 7 templates.

Le callback Mobile Money /api/payments/mobile-money/callback verifie correctement la signature HMAC-SHA256 en mode production, mais le mode sandbox retourne systematiquement true. Le webhook WhatsApp n'a aucune verification de signature ni validation du hub.verify_token.

8. Build et Tests

Le build de production reussit sans erreur TypeScript. 352 tests passent sur 13 suites de test. La couverture couvre le service OTP, le fournisseur SMS, les schemas de validation, la securite, les routes API d'authentification et Redis. Les tests ne couvrent PAS les routes CRUD cliniques (patients, consultations, etc.) ni les integrations. Le pipeline CI/CD GitHub Actions est configure avec lint, test, build, security scan et deploy. Sentry est configure pour le monitoring client+serveur.

9. Plan d'Action Priorise

9.1 Actions immediates (avant production)

#	Action	Severite	Fichier(s)
1	Corriger la divergence JWT_SECRET dans patient-auth/me	CRITIQUE	patient-auth/me/route.ts

2	Ajouter secureApiHandler a tous les endpoints CRUD	CRITIQUE	46 fichiers route
3	Ajouter validation Zod a tous les POST/PUT	CRITIQUE	40+ handlers
4	Appliquer RBAC sur toutes les routes API	CRITIQUE	api-middleware.ts
5	Regenerer les secrets (NEXTAUTH, JWT, ENCRYPTION)	CRITIQUE	.env
6	Mettre DEMO_MODE=false en production	HAUT	.env
7	Ajouter rate limiting sur auth/login et messaging	HAUT	routes specifiques
8	Valider le hub.verify_token du webhook WhatsApp	HAUT	messaging/whatsapp
9	Remplacer Math.random() par crypto.randomInt()	HAUT	6+ fichiers
10	Corriger CORS .includes() par URL parsing	HAUT	middleware.ts

Tableau 5 : Plan d'action immediat

9.2 Actions a court terme (1 sprint)

Migrer les stores Zustand vers des appels API reels (priorite 1 frontend). Ajouter les indexes manquants sur establishmentId pour 7+ modeles. Ajouter les regles onDelete sur les 42 relations critiques. Convertir les 56 champs String en enums Prisma. Ajouter les champs createdBy/updatedAt aux modeles medicaux. Migrer le rate limiting du middleware vers Redis. Appliquer les filtres RLS aux requetes de donnees patient. Ajouter des error boundaries a toutes les pages. Completer les icones PWA et les traductions i18n.

9.3 Actions a moyen terme (backlog)

Standardiser les formats de reponse API (successResponse/paginatedResponse). Nettoyer les console.log en production (~70 instances). Reduire les types any (~40 instances). Deplacer les tokens du localStorage vers des cookies httpOnly. Ajouter la verification de signature JWT dans verifyJwtSession avec jose.jwtVerify. Completer les traductions Fulfulde et Susu (27%). Nettoyer les fichiers orphelins (.deb, db/custom.db, upload/). Mettre a jour .env.example.